ALGORITMOS E ESTRUTURAS DE DADOS I

Pilhas em C utilizando Listas Encadeadas

1 O problema

Desenvolva um programa em C que implemente uma pilha de números inteiros utilizando uma lista simplesmente encadeada.

O programa deverá disponibilizar um menu interativo para que o usuário possa:

- Inserir um elemento na pilha;
- Remover o elemento do topo;
- Consultar o elemento do topo;
- Verificar se a pilha está vazia;
- Exibir todos os elementos;
- Consultar a quantidade de elementos;
- Liberar toda a memória utilizada.

Todas as operações deverão respeitar a política LIFO.

2 Fundamentação teórica

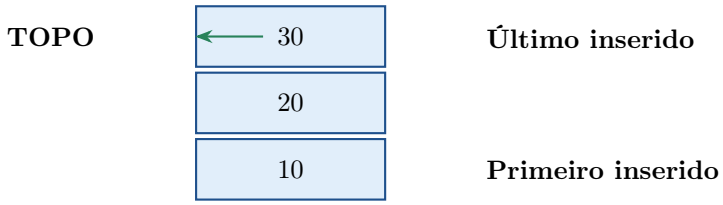
2.1 O que é uma pilha?

Uma pilha é uma estrutura na qual as operações de inserção e remoção acontecem em uma única extremidade, denominada **topo**.

Considere a seguinte sequência:

1. Inserir o valor 10;
2. Inserir o valor 20;
3. Inserir o valor 30.

A pilha ficará representada da seguinte maneira:



O valor 30 está no topo porque foi o último elemento inserido.

Caso seja realizada uma operação de remoção, o valor 30 será retirado primeiro.

2.2 Política LIFO

A política LIFO determina que:

Último a entrar, primeiro a sair

Essa característica diferencia as pilhas de outras estruturas de dados, como as filas, que seguem a política FIFO.

3 Implementação utilizando lista encadeada

Uma pilha pode ser implementada utilizando uma lista simplesmente encadeada.
Cada nó armazenará:

- Um valor inteiro;
- Um ponteiro para o próximo nó.

A estrutura deverá ser semelhante à seguinte:

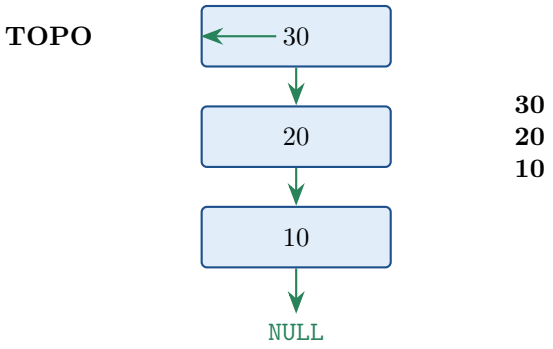
```c
typedef struct No {
    int dado;
    struct No *proximo;
} No;
```

A pilha será representada por um ponteiro chamado `topo`, que indica o primeiro nó da lista.

```c
typedef struct Pilha {
    No *topo;
} Pilha;
```

3.1 Representação gráfica

Considere uma pilha contendo os valores 10, 20 e 30.



Observe que o ponteiro `topo` aponta para o nó que contém o valor 30.
Cada nó aponta para o elemento inserido anteriormente.

4 Operações obrigatórias

O grupo deverá implementar todas as operações descritas nesta seção.

4.1 1. Inicialização da pilha

Criar uma função responsável por inicializar a pilha.
Uma pilha recém-criada deverá estar vazia.

```c
void inicializar(Pilha *pilha);
```

O ponteiro `topo` deverá receber `NULL`.

4.2 2. Verificar se a pilha está vazia

Implementar uma função que verifique se existe algum elemento armazenado.

```c
int estaVazia(Pilha *pilha);
```

A função deverá retornar:

- 1, caso a pilha esteja vazia;
- 0, caso exista pelo menos um elemento.

4.3 3. Empilhar (PUSH)

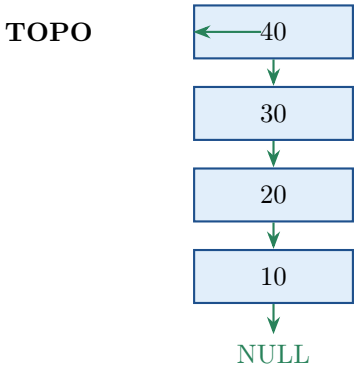
A operação `push` consiste em inserir um novo elemento no topo da pilha.
Considere inicialmente:

$$30 \rightarrow 20 \rightarrow 10 \rightarrow NULL$$

Ao executar `push(40)`, a pilha deverá ficar:

$$40 \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow NULL$$

4.3.1 Representação gráfica



O grupo deverá implementar:

```c
void push(Pilha *pilha, int valor);
```

A função deverá:

1. Alocar memória para um novo nó;
2. Verificar se a alocação foi realizada;
3. Armazenar o valor recebido;
4. Fazer o novo nó apontar para o antigo topo;
5. Atualizar o ponteiro `topo`.

4.4 4. Desempilhar (POP)

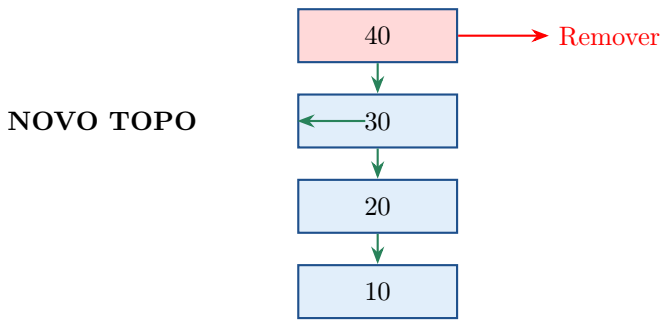
A operação `pop` consiste em remover o elemento que está no topo da pilha.
Considere:

$$40 \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow NULL$$

Após executar `pop()`, a pilha deverá ficar:

$$30 \rightarrow 20 \rightarrow 10 \rightarrow NULL$$

4.4.1 Representação gráfica



O grupo deverá implementar:

```c
int pop(Pilha *pilha);
```

A função deverá:

1. Verificar se a pilha está vazia;
2. Guardar o endereço do nó que será removido;
3. Atualizar o ponteiro `topo`;
4. Liberar a memória do nó removido;
5. Retornar o valor removido.

Caso a pilha esteja vazia, a função deverá tratar a situação sem acessar memória inválida.
O grupo deverá definir e justificar uma estratégia para indicar que não foi possível desempilhar, sem confundir essa situação com um valor válido armazenado na pilha.

4.5 5. Consultar o topo (PEEK ou TOP)

Implementar uma função que consulte o valor do topo sem remover o elemento.

```c
int peek(Pilha *pilha, int *valor);
```

A função deverá:

- Verificar se a pilha está vazia;
- Retornar o valor armazenado no topo;
- Não modificar a estrutura da pilha.

O parâmetro `valor` poderá ser utilizado para retornar o elemento consultado, enquanto o retorno da função indicará se a operação foi realizada.

4.6 6. Exibir os elementos

Implementar uma função que percorra a pilha e imprima todos os elementos, começando pelo topo.

```c
void imprimirPilha(Pilha *pilha);
```

Exemplo de saída:

Exemplo

```
TOPO
  |
  v
  40
  30
  20
  10
  |
 NULL
```

A função deverá utilizar um ponteiro auxiliar, sem alterar o ponteiro `topo`.

4.7 7. Contar elementos

Implementar uma função que retorne a quantidade de elementos armazenados na pilha.

```c
int contarElementos(Pilha *pilha);
```

4.8 8. Liberar a pilha

Implementar uma função que libere todos os nós alocados dinamicamente.

```c
void liberarPilha(Pilha *pilha);
```

Após a execução, a pilha deverá estar vazia, com o ponteiro `topo` igual a `NULL`.

5 Estrutura inicial do código

O grupo deverá utilizar a estrutura apresentada a seguir como ponto de partida.

As funções deverão ser implementadas pelos próprios estudantes.

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

typedef struct Pilha {
    No *topo;
} Pilha;

// Inicializar a pilha
void inicializar(Pilha *pilha);

// Verificar se esta vazia
int estaVazia(Pilha *pilha);

// Inserir elemento (PUSH)
void push(Pilha *pilha, int valor);

// Remover elemento (POP)
int pop(Pilha *pilha);

// Consultar topo (PEEK)
int peek(Pilha *pilha, int *valor);

// Exibir elementos
void imprimirPilha(Pilha *pilha);

// Contar elementos
int contarElementos(Pilha *pilha);

// Liberar memoria
void liberarPilha(Pilha *pilha);

int main() {

    Pilha pilha;

    inicializar(&pilha);

    // Implementar menu e chamadas das funcoes

    liberarPilha(&pilha);

    return 0;
}
```

6 Testes obrigatórios

O programa deverá ser testado considerando pelo menos as seguintes situações:

Teste	Situação	Resultado esperado
1	Inicializar pilha	Pilha vazia
2	Executar PUSH(10)	Topo igual a 10
3	Executar PUSH(20)	Topo igual a 20
4	Executar PUSH(30)	Topo igual a 30
5	Executar POP()	Remover 30
6	Executar PEEK()	Consultar 20, sem remover
7	Exibir elementos	20 seguido de 10
8	Contar elementos	Retornar 2
9	Desempilhar todos os elementos	Pilha vazia
10	Executar POP() em pilha vazia	Tratar operação inválida
11	Executar PEEK() em pilha vazia	Informar que não há elemento
12	Liberar pilha	Todos os nós liberados

Os testes deverão contemplar também:

- Inserção de apenas um elemento;
- Inserção de vários elementos;
- Remoção sucessiva de elementos;
- Consulta ao topo sem alteração da pilha;
- Inserção após a pilha ficar vazia;
- Tentativa de remover elementos de uma pilha vazia.

7 Análise de complexidade

O grupo deverá analisar a complexidade temporal de cada operação implementada.

A análise deverá considerar uma pilha implementada com lista simplesmente encadeada e um ponteiro para o topo.

O grupo deverá preencher a tabela:

Operação	Complexidade	Justificativa
Inicialização		
PUSH		
POP		
PEEK		
Verificar se está vazia		
Imprimir elementos		
Contar elementos		
Liberar memória		

Além disso, o grupo deverá explicar por que as operações de inserção e remoção no topo podem ser realizadas sem percorrer toda a lista.

8 Perguntas para discussão e defesa

As perguntas a seguir poderão ser utilizadas durante o seminário.

1. Por que uma pilha segue a política LIFO?
2. Qual a função do ponteiro `topo`?
3. Por que a operação PUSH pode ser realizada em tempo constante?
4. Por que a operação POP também pode ser realizada em tempo constante?

5. O que acontece com o ponteiro `topo` quando o último elemento é removido?
6. Qual a diferença entre consultar o topo e desempilhar um elemento?
7. O que aconteceria se o programa tentasse executar POP em uma pilha vazia?
8. Por que devemos guardar o endereço do nó removido antes de atualizar o ponteiro `topo`?
9. O que acontece se utilizarmos `free()` antes de atualizar o ponteiro `topo`?
10. Qual a vantagem de utilizar alocação dinâmica na implementação da pilha?
11. Qual a diferença entre uma pilha implementada com vetor e uma pilha implementada com lista encadeada?
12. É possível implementar uma pilha utilizando uma lista duplamente encadeada? Seria necessário?
13. Como uma pilha pode ser utilizada para verificar parênteses balanceados?
14. Por que a operação de impressão possui complexidade diferente das operações PUSH e POP?

9 Reflexão final

Após concluir a atividade, responda:

1. Por que a política LIFO é importante para o funcionamento de uma pilha?
2. Em quais situações do cotidiano ou da computação uma pilha pode ser utilizada?
3. Quais são as vantagens de implementar uma pilha utilizando listas encadeadas?
4. Qual foi a maior dificuldade encontrada durante a implementação e como ela foi resolvida?
5. Como a utilização de uma pilha pode facilitar a implementação de sistemas que precisam desfazer operações?